Time Complexity 04

Thumrongsak Kosiyatrakul
tkosiyat@cs.pitt.edu

NP-COMPLETENESS

- **NP-Complete** is a class of problems discovered by Cook and Levin
  - Complexity of these problems are closely related
  - If a polynomial time algorithm exists for a problem in this class, problems of entire class can be solved in polynomial time

Polynomial Time Reducibility

Definition 7.28

A function $f : \Sigma^* \to \Sigma^*$ is a **polynomial time computable function** if some polynomial time Turing machine M exists that halts with just $f(w)$ on its tape, when started on any input w.

Definition 7.29

Language A is **polynomial time mapping reducible**, or simply **polynomial time reducible**, to language B, written $A \leq_p B$, if a polynomial time computable function $f : \Sigma^* \to \Sigma^*$ exists, where for every w,

$$w \in A \leftrightarrow f(w) \in B.$$

The function f is called the **polynomial time reduction** of A to B.

Polynomial Time Reducibility

- Recall the property from previous slide:

$$w \in A \leftrightarrow f(w) \in B$$

- This is exactly the same as **mapping reducibility** property
- Recall that if $A \leq_m B$ and B is decidable, A is decidable
- $A \leq_p B$ gives you exactly the same as $A \leq_m B$ with one extra property that the reduction f runs in polynomial time

Polynomial Time Reducibility

Theorem 7.31

If $A \leq_p B$ and $B \in P$, then $A \in P$.

- If you can **convert** an instance of a problem A into an instance of a problem B in **polynomial time**
- If the problem B can be **solved in polynomial time**
- The problem A can be **solved in polynomial time**

Polynomial Time Reducibility

Theorem 7.31

If $A \leq_p B$ and $B \in P$, then $A \in P$.

Proof:

- Assume that $A \leq_p B$
  - There exists a **polynomial time** reduction f satisfying:

$$w \in A \leftrightarrow f(w) \in B$$

- Assume that $B \in P$
  - There exists a TM M_B that decides B
  - M_B runs in polynomial time
- We need to show that we can decide A in polynomial time
  - Need a TM M_A that decides A in polynomial time

Polynomial Time Reducibility

- This TM M_A decides A:

  $M_A =$ "On input w:

  1. Compute $f(w)$ where f is a polynomial time reduction from A to B.
  2. Run M_B (a decider of B) on $f(w)$.
  3. If M_B accepts $f(w)$, *accept*. If M_B rejects $f(w)$, *reject*."
- Prove that M_A is a decider for A:
  - $w \in A \rightsquigarrow f(w) \in B \rightsquigarrow M_B$ accepts $f(w) \rightsquigarrow M_A$ accepts w
  - $w \notin A \rightsquigarrow f(w) \notin B \rightsquigarrow M_B$ rejects $f(w) \rightsquigarrow M_A$ rejects w
- Since f runs in polynomial time and M_B can decide B in polynomial time, M_A runs in polynomial time
- Since M_A decides A in polynomial time, $A \in P$

Satisfiability Problem

- A formula ϕ can be construed from **boolean variables** and **boolean operators** ($\wedge$, $\vee$, and $\neg$)
  - Sometimes we use $\overline{x}$ instead of $\neg x$ for a boolean variable x
  - For simplicity, this textbook uses 1 for true and 0 for false
- For example, $\phi = (\overline{x} \wedge y) \vee (x \wedge \overline{z})$
- A formula is **satisfiable** if there exists an assignment of 0s and 1s to the variables which makes the formula evaluate to 1
  - In the above example, ϕ is 1 when $x = 0$, $y = 1$, and $z = 0$.
- The **satisfiability problem** is defined formally as follows:

$$SAT = \{\langle \phi \rangle \mid \phi \text{ is a satisfiable Boolean formula}\}$$

Conjunctive Normal Form (CNF)

- We will use a special form of boolean formula called **Conjunctive Normal Form** (cnf)
- A **literal** is a Boolean variable or a negation of a boolean variable, as in x or $\overline{x}$
- A **clause** is several literals connected with $\vee$s

$$(x_1 \vee \overline{x_2} \vee \overline{x_3} \vee x_4)$$

- A **cnf-formula** comprises several clauses connected with $\wedge$s

$$(x_1 \vee \overline{x_2} \vee \overline{x_3} \vee x_4) \wedge (x_3 \vee \overline{x_5} \vee x_6) \wedge (x_3 \vee \overline{x_6})$$

- A **3cnf-formula** is a cnf-formula where all the clauses have exactly three literals, as in

$$(x_1 \vee \overline{x_2} \vee \overline{x_3}) \wedge (x_3 \vee \overline{x_5} \vee x_6) \wedge (x_3 \vee \overline{x_6} \vee x_4) \wedge (x_4 \vee x_5 \vee x_6)$$

- Let $3SAT = \{\langle \phi \rangle \mid \phi \text{ is a satisfiable 3cnf-formula}\}$

$3SAT \leq_p CLIQUE$

- Is $3SAT \leq_p CLIQUE$?
- We need to create a **polynomial time reduction** $f : \Sigma^* \to \Sigma^*$ such that

$$w \in 3SAT \leftrightarrow f(w) \in CLIQUE$$

- Recall $3SAT$ and $CLIQUE$:

$$3SAT = \{\langle \phi \rangle \mid \phi \text{ is a satisfiable 3cnf-formula}\}$$
$$CLIQUE = \{\langle G, k \rangle \mid G \text{ is an undirected graph with a } k\text{-clique}\}$$

- These are two totally different problems
- We need to convert $\langle \phi \rangle$ to $\langle G, k \rangle$ such that ϕ is satisfiable if and only if graph G contains k-clique for some k

$3SAT \leq_p CLIQUE$

- A 3cnf-formula is of the form $\phi = c_1 \wedge c_2 \wedge \ldots c_n$ for some $n \geq 1$
- ϕ is evaluated to 1 if $c_i = 1$ for every $1 \leq i \leq n$
- Every clause c_i in ϕ depends on other clauses c_j in ϕ to be 1 to make ϕ evaluated to 1
- Each clause c in a 3cnf-formula is of the form

$$c = (x_p \vee x_q \vee x_r)$$

  for literals x_p, x_q, and x_r
- The clause c is evaluated to 1 if one of its literal is 1
- If a literal in a clause is 1, for ϕ to be evaluated to 1, it depends on at least one literal in other clauses to be 1
  - Note that it cannot depend on its negation literals
  - It does not depend on the other two literals in its clause since its clause is already evaluated to 1
- This create relations among n literals (one from each clause) which can be represented using edges in an indirected graph

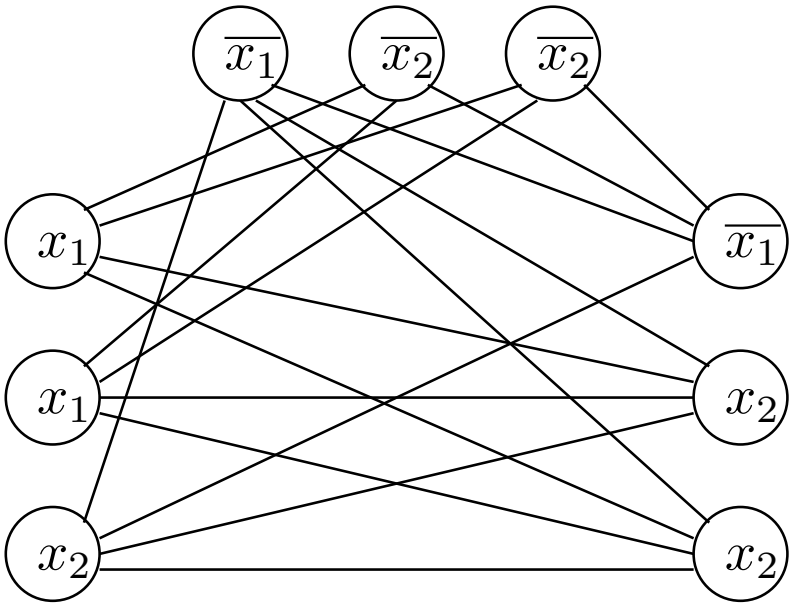
$3SAT \leq_p CLIQUE$

- Let ϕ be a formula with k clauses as

$$\phi = (a_1 \vee b_1 \vee c_1) \wedge (a_2 \vee b_2 \vee c_2) \wedge \cdots \wedge (a_k \vee b_k \vee c_k)$$

- Nodes in G are organized in k groups of tree nodes each call triples
- Each triple corresponds to one of the clauses in ϕ
- Each node in a triple corresponds to a literal in its associated clause
- Edges connect every two nodes in G except the following:
  - No edge among nodes in the same triple
  - No edge is present between two nodes with contradictory labels, as in x_2 and $\overline{x_2}$

$3SAT \leq_p CLIQUE$



$$\phi = (x_1 \vee x_1 \vee x_2) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_2 \vee x_2)$$

- $3SAT \leq_p CLIQUE$
- If a boolean 3cnf formular ϕ with k clauses is satisfiable, the graph converted from ϕ will contain k clique

$3SAT \leq_p CLIQUE$

- If ϕ is satisfiable, there exists an assignment that makes $\phi = 1$
  - For each clause, select exactly one true literal based on the assignment
  - If there are k clauses, selected literals form k-clique
- If graph G has a k-clique
  - Each node in k-clique corresponds to a node (literal) in a triple (clause)
  - Assign true to those literals
- Recall the theorem 7.31:

$$\text{If } A \leq_p B \text{ and } B \in P, \text{ then } A \in P$$

- If we can solve $CLIQUE$ in polynomial time, we can solve $3SAT$ in polynomial time by simply convert $\langle \phi \rangle$ to $\langle G, k \rangle$ (in polynomial time) and then solve it

NP-Completeness

- Recall that NP-Complete is a class of problems that are closely related
- If you can solve one problem in this class in polynomial time, all problems in this class can be solved in polynomial time

Definition 7.34

A language B is **NP-Complete** if it satisfies two conditions:

1. B is in NP, and
2. Every language A in NP is polynomial time reducible to B.

NP-Completeness

- In other words, to show that a language B is **NP-Complete** we need to show the following:
  1. Show that B is in NP which can be done by one of the following:
     1. Show a polynomial time verifier V for B, or
     2. Show a NTM N that decide B in polynomial time
  2. Show that every language A in NP is polynomial time reducible to B
     - Show an **polynomial time** reduction $f : \Sigma^* \to \Sigma^*$ satisfying

$$w \in A \leftrightarrow f(w) \in B$$

for every language $A \in NP$

SAT is NP-Complete

- So, to show that SAT is NP-Complete, we need to show the following:
  1. $SAT \in NP$
     - Simply construct a NTM that generates all possible assignments
     - If one of them evaluated to 1, $accept$
  2. Show that a every language A in NP is polynomial time reducible to SAT
     - Unfortunately, there are a lot of languages in NP
     - But if we have an NP-Complete language B
       $\rightsquigarrow$ B is in NP
       $\rightsquigarrow$ Every language $A \in NP$, $A \leq_p B$
     - If we can show that $B \leq_p SAT$, every language $A \in NP$, $A \leq_p SAT$
     - But we do not have an NP-Complete problem yet

SAT is NP-Complete

- Suppose a language A is in NP
  - There exists a NTM M that decides A in polynomial time
- Note that we do not specify what is the language A and what is the description of NTM M
  - The language A can be any language that is in NP and M is an NTM that decides A in polynomial time
- What we need is to show that $A \leq_p SAT$?
  - Since we do not know what is the language A, we cannot simply use the description of A
  - Even though we do not have the description of NTM M but we know the following:
    - M is a nondeterministic Turing machine
    - NTM M is a decider for the language A

SAT is NP-Complete

- About NTM M that decides A
  - If M accepts $w \rightsquigarrow w \in A \rightsquigarrow$ the accepting computation history for M on input w exists
  - If M rejects $w \rightsquigarrow w \notin A \rightsquigarrow$ the accepting computation history for M on input w does not exists
- This is an NTM, there can be multiple computation histories
- We need to use **reduction via computation histories**
  - Convert computation histories of M on input w to a formula ϕ such that
    - If the accepting computation history for M on input w exists, ϕ is satisfiable ($\phi \in SAT$)
    - If the accepting computation history for M on input w does not exists, ϕ is not satisfiable ($\phi \notin SAT$)
- Cook-Levin Theorem shows the method describe above and conclude that SAT is NP-Complete.

$3SAT$ is NP-Complete

- Since we already know that SAT is NP-Complete, to show that $3SAT$ is NP-Complete, we need to show the following:
  1. $3SAT \in NP$:
     - Construct an NTM that generates all possible assignments
     - If one of them evaluate to 1, $accept$
  2. $SAT \leq_p 3SAT$:
     - Convert the formula ϕ into a 3cnf $f(\phi)$ satisfying

$$\phi \in SAT \leftrightarrow f(\phi) \in 3SAT$$

where f is a **polynomial time** reduction from SAT to $3SAT$

From SAT to $3SAT$

- If a clause has less than three literal, add one or two literals from the same clause
  - $x_1 \rightsquigarrow (x_1 \vee x_1 \vee x_1)$
  - $(x_1 \vee \overline{x_2}) \rightsquigarrow (x_1 \vee x_1 \vee \overline{x_2})$
  - $(x_1 \vee \overline{x_2}) \rightsquigarrow (x_1 \vee \overline{x_2} \vee \overline{x_2})$
  - These results are logically equivalent with their original clauses
- If a clause has more than tree literals, split it into a number of clauses of three literals each.

$$(x_1 \vee x_2 \vee x_3 \vee x_4) \rightsquigarrow (x_1 \vee x_2 \vee z) \wedge (\overline{z} \vee x_3 \vee x_4)$$

  - This is not logically equivalent
  - It is equivalent in terms of satisfiability

Additional NP-Complete Problems

- $CLIQUE$ is NP-Complete
  - We already show that $3SAT \leq_p CLIQUE$
- $VERTEX-COVER$ is NP-Complete where

  $VERTEX-COVER = \{\langle G, k\rangle \mid G$ is an undirected graph that has a k-node vertex cover$\}$

  - Textbook shows that $3SAT \leq_p VERTEX-COVER$
- $HAMPATH$ is NP-Complete
  - Textbook shows that $3SAT \leq_p HAMPATH$
- $SUBSET-SUM$ is NP-Complete
  - Textbook shows that $3SAT \leq_p SUBSET-SUM$